

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Конструкторская часть	4
1.1 Анализ требований технического задания	4
1.2 Разработка структурной схемы	5
1.3 Обоснование выбора элементной базы	5
1.3.1 Выбор микроконтроллера	6
1.3.2 Выбор датчика температуры	6
1.3.3 Выбор драйвера двигателя	7
1.3.4 Выбор средств индикации	8
1.3.5 Выбор преобразователя интерфейса	8
1.4 Разработка функциональной схемы	9
1.5 Описание микроконтроллера ATmega8515	11
1.6 Принцип работы и DRV8871	12
1.7 Описание принципиальной электрической схемы	14
1.8 Описание программной реализации и алгоритмов	17
1.8.1 Алгоритм основного цикла	17
1.8.2 Алгоритм обработки обновления состояния вентилятора	18
1.8.3 Алгоритм обработки команды UART	19
1.9 Питание устройства	20
1.10 Интерфейс программирования	21
1.11 Расчетная часть	22
1.11.1 Расчет элементов драйвера двигателя (DRV8871)	22
1.11.2 Расчет энергопотребления и выбор источника питания	28
1.11.3 Потребление по цепи +12 В	28
1.11.4 Общее потребление устройства	29
1.11.5 Выбор источника питания	29
2 Технологическая часть	30
2.1 Работа в системах разработки и отладки	30
2.2 Структура программного обеспечения	31
2.3 Настройка периферийных устройств	32

2.4 Макетная отладка и программирование микроконтроллера	32
2.5 Отладка и тестирование	33
2.6 Методика тестирования и проверочные тесты	33
ЗАКЛЮЧЕНИЕ	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37
ПРИЛОЖЕНИЕ А ИСХОДНЫЙ ТЕКСТ ПРОГРАММЫ	38

ВВЕДЕНИЕ

Системы активного охлаждения являются неотъемлемой частью современной электроники, обеспечивая отвод тепла от греющихся компонентов. Отсутствие интеллектуального управления приводит к повышенному шуму, износу вентиляторов и нерациональному расходу электроэнергии.

Целью данной работы является разработка автоматизированной системы управления вентилятором на базе 8-битного микроконтроллера, способной работать как автономно, так и под внешним управлением.

Для достижения цели решены следующие задачи: – обоснован выбор элементной базы (МК ATmega8515, датчик DHT11, драйвер DRV8871);

- разработана схема электрическая принципиальная;
- реализованы алгоритмы считывания данных по нестандартному протоколу, генерации ШИМ и работы с энергонезависимой памятью;
- выполнена программная реализация устройства на языке C.

1 Конструкторская часть

1.1 Анализ требований технического задания

Согласно техническому заданию, необходимо разработать на основе микроконтроллера устройство автоматизированного управления системой активного охлаждения.

Устройство должно обеспечивать регулирование работы вентилятора в зависимости от текущих показаний датчика температуры. Для исключения частого включения и выключения двигателя при колебаниях температуры около порогового значения необходимо реализовать алгоритм гистерезиса. Также устройство должно предоставлять возможность ручного управления скоростью вращения вентилятора посредством ШИМ.

Важным требованием является реализация двустороннего обмена данными и командами управления по протоколу UART. Микроконтроллер должен принимать команды настройки, например, изменение температурных порогов или скорости и передавать информацию о статусе системы внешнему устройству. Для отображения этой информации непосредственно на устройстве необходимо использовать символьный жидкокристаллический дисплей.

Для обеспечения сохранения пользовательских настроек при отключении питания требуется использование энергонезависимой памяти EEPROM.

Поскольку вентилятор является индуктивной нагрузкой и потребляет ток, значительно превышающий нагрузочную способность выводов микроконтроллера, необходимо использовать силовой драйвер двигателя. Драйвер должен обеспечивать согласование уровней напряжения и тока, а также поддерживать управление скоростью через ШИМ.

Для подключения к персональному компьютеру, который зачастую не имеет нативного COM-порта, устройство целесообразно снабдить преобразователем интерфейсов USB-UART или соответствующим разъемом для подключения внешнего адаптера.

1.2 Разработка структурной схемы

Структурная схема устройства разработана на основе анализа технического задания и определяет основные функциональные узлы системы. Согласно требованиям, устройство состоит из следующих блоков:

- МК — центральное устройство управления, осуществляющее обработку данных и формирование управляющих сигналов;
- датчик — цифровой модуль измерения температуры и влажности;
- мотор — исполнительный механизм (вентилятор) системы охлаждения;
- дисплей — средство визуального отображения текущих параметров системы;
- кнопки — интерфейс пользователя для ручной настройки параметров;
- разъем подключения к ПК — обеспечивает физическое соединение для обмена данными.

На рисунке 1 представлена структурная схема устройства.

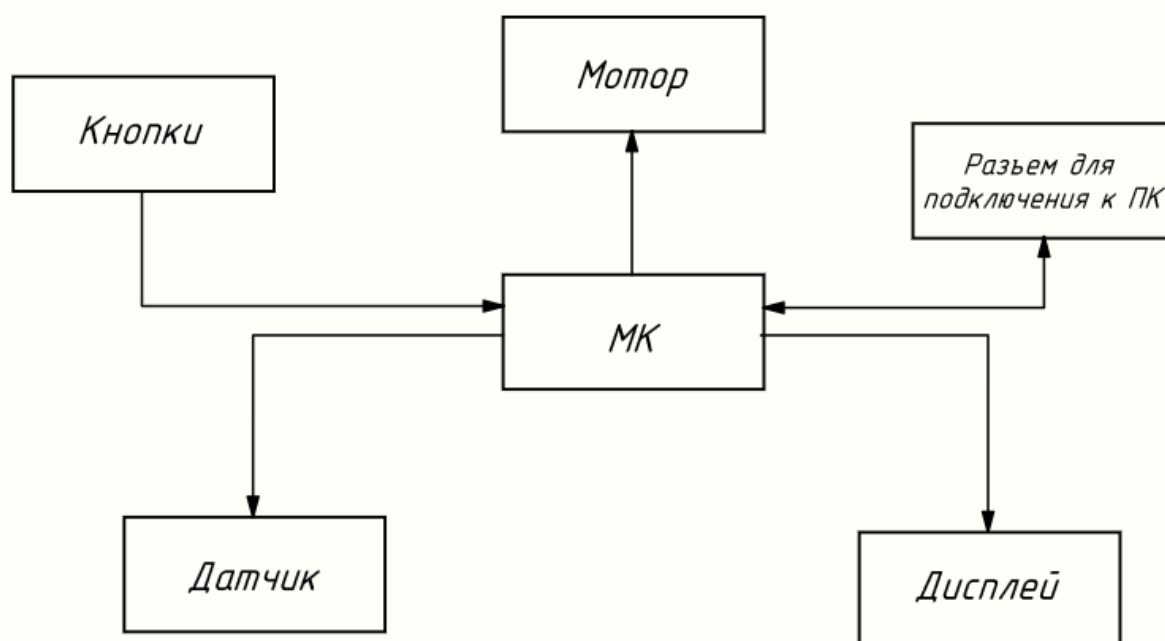


Рисунок 1 — Структурная схема

1.3 Обоснование выбора элементной базы

Выбор компонентов производился на основе анализа технических требований, сравнительных характеристик аналогов, доступности и удобства монтажа.

1.3.1 Выбор микроконтроллера

В качестве управляющего устройства рассматривались 8-битные микроконтроллеры архитектуры AVR. Основными критериями выбора являлись: объем памяти программ Flash и данных EEPROM, количество портов ввода-вывода GPIO и наличие необходимых аппаратных интерфейсов: UART, Timer/Counter. Сравнительный анализ представлен в таблице 1.

Таблица 1 — Сравнительные характеристики микроконтроллеров

Параметр	ATmega8515	ATmega8	ATmega328P
Flash-память, КБ	8	8	32
EEPROM, байт	512	512	1024
SRAM, байт	512	1024	2048
Линии ввода-вывода	35	23	23
Таймеры (8/16 бит)	1 / 1	2 / 1	2 / 1
Корпус	DIP-40	DIP-28	DIP-28

Для реализации проекта выбран микроконтроллер ATmega8515 [1]. Несмотря на меньший объем SRAM по сравнению с аналогами, его ресурсов достаточно для решения поставленной задачи. Наличие 512 байт EEPROM удовлетворяет требованиям по хранению настроек.

1.3.2 Выбор датчика температуры

Для мониторинга параметров окружающей среды рассматривались цифровые и аналоговые датчики. Сравнение приведено в таблице 2.

Таблица 2 — Сравнительные характеристики датчиков

Параметр	DHT11	DS18B20	LM35
Тип	Цифровой	Цифровой	Аналоговый
Измеряемые величины	T + H	T	T
Диапазон T, °C	0...50	-55...+125	-55...+150
Точность T, °C	±2	±0.5	±0.5

Продолжение таблицы 2

Интерфейс	1-Wire	1-Wire	АЦП
-----------	--------	--------	-----

Выбран датчик DHT11 [2]. Устройство предназначено для управления вентиляцией в жилых помещениях, где диапазон 0...50 °С является достаточным. Важным преимуществом DHT11 является возможность измерения влажности, что расширяет функционал системы. Использование цифрового интерфейса исключает необходимость калибровки АЦП и снижает влияние помех на линии связи.

1.3.3 Выбор драйвера двигателя

Для управления двигателем постоянного тока необходим драйвер, обеспечивающий усиление по току и возможность регулирования скорости. Сравнение вариантов приведено в таблице 3.

Таблица 3 — Сравнительные характеристики драйверов

Параметр	DRV8871	L298N	Дискр. MOSFET
Тип	Н-мост	Н-мост	Ключ
Макс. ток, А	3.6	2.0	>10
Напряжение, В	6.5...45	до 46	Зависит от типа
Защита	ОСР, TSD, UVLO	Нет	Внешняя
КПД	Высокий	Низкий	Высокий

Выбрана микросхема DRV8871 [3]. В отличие от устаревшего L298N (биполярная технология), DRV8871 выполнен на полевых транзисторах (MOSFET). Наличие встроенных аппаратных защит от перегрузки по току (ОСР) и перегрева (TSD) повышает надежность устройства и упрощает схемотехнику, исключая внешние защитные компоненты.

1.3.4 Выбор средств индикации

Для отображения информации пользователю рассматривались различные типы дисплеев.

Таблица 4 — Сравнение средств индикации

Параметр	LCD 1602 (LM016L)	OLED 0.96"
Тип информации	Символьный	Графический
Интерфейс	Параллельный 4- бит	I2C/SPI
Читаемость	Высокая	Высокая
Потребление	Среднее	Низкое

Выбран жидкокристаллический модуль LM016L [4]. Он позволяет выводить текстовую информацию (статус режима, единицы измерения), что делает интерфейс интуитивно понятным по сравнению с 7-сегментными индикаторами. Параллельный интерфейс легко реализуется программно на выбранном микроконтроллере.

1.3.5 Выбор преобразователя интерфейса

Для подключения микроконтроллера (UART TTL) к порту USB персонального компьютера необходим преобразователь интерфейсов.

Таблица 5 — Сравнение преобразователей USB-UART

Параметр	CH340N	FT232RL	CP2102
Корпус	SOP-8	SSOP-28	QFN-28
Обвязка	Минимальная	Средняя	Средняя
Встроенный генератор	Да	Да	Да

Выбрана микросхема CH340N [5]. Она выпускается в компактном корпусе SOP-8 и требует минимального количества внешних компонентов, так как имеет встроенный тактовый генератор. Это упрощает разводку печатной платы и снижает стоимость устройства.

1.4 Разработка функциональной схемы

Функциональная схема устройства разработана на основе структурной схемы и выбранной элементной базы. Она детализирует внутреннюю архитектуру микроконтроллера, распределение его логических ресурсов и организацию связей с периферийными модулями.

Схема электрическая функциональная приведена на рисунке 2.

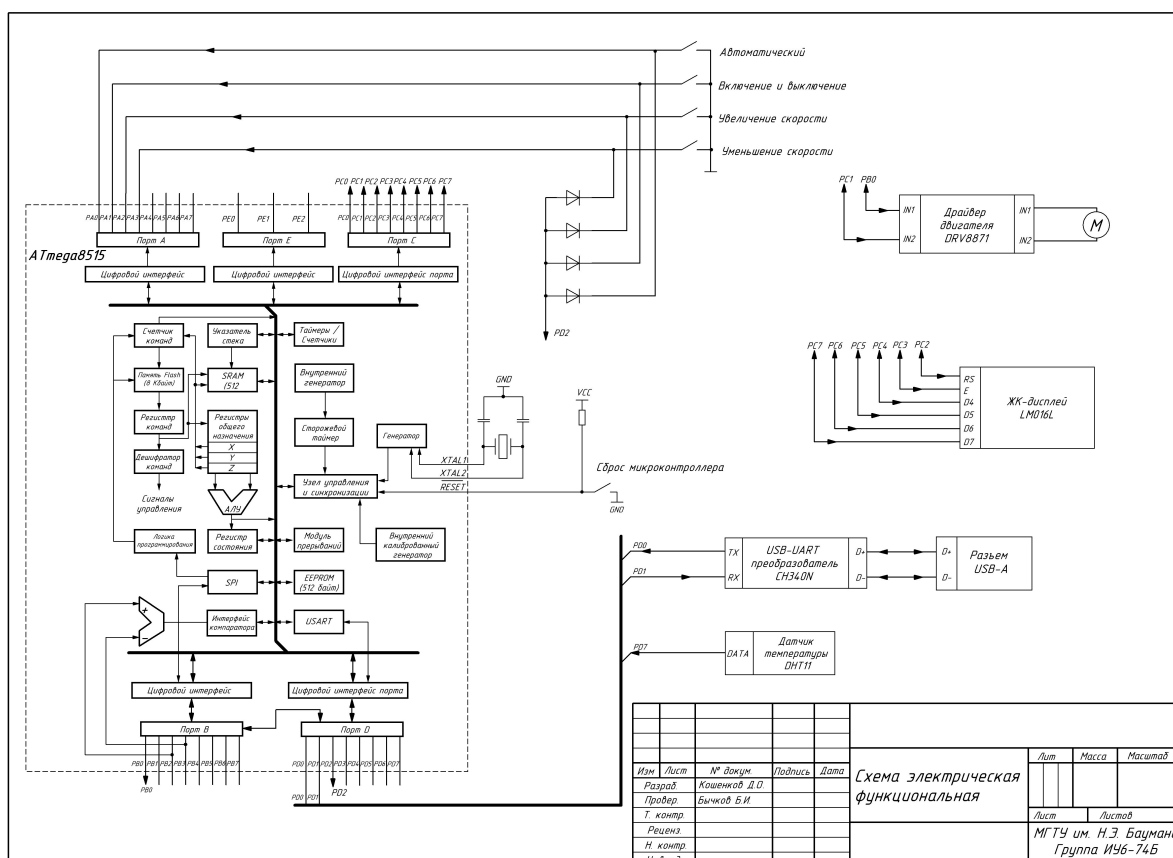


Рисунок 2 — Схема электрическая функциональная

Ядром системы является микроконтроллер ATmega8515, который на схеме представлен в виде совокупности функциональных блоков: арифметико-логического устройства, подсистемы памяти, включающей Flash-память программ, оперативное запоминающее устройство и энергонезависимую память EEPROM. Взаимодействие с внешним миром и внутренними процессами обеспечивают периферийные модули, такие как таймеры-счетчики, контроллер прерываний, последовательные интерфейсы SPI и USART, а также порты ввода-вывода.

Синхронизация работы вычислительного ядра осуществляется системой тактирования, построенной на базе внешнего кварцевого резонатора и

внутреннего генератора микроконтроллера. Такое решение гарантирует высокую стабильность частоты, необходимую для корректной передачи данных по временным протоколам. Схема сброса обеспечивает надежную инициализацию устройства при подаче питания или по внешнему сигналу.

Блок пользовательского ввода состоит из группы коммутационных элементов, подключенных к линиям порта A. Сигналы от кнопок дополнительно объединены логической схемой для формирования запроса внешнего прерывания. Это позволяет микроконтроллеру мгновенно реагировать на действия оператора, выходя из режима ожидания или прерывая текущую задачу для обработки команды управления.

Система визуализации реализована на символьном жидкокристаллическом дисплее. Обмен информацией с индикатором происходит через порт C, где часть линий используется для параллельной передачи данных, а отдельные линии выделены для управляющих сигналов синхронизации и выбора режима работы дисплея.

Исполнительный узел управления вентилятором включает специализированный драйвер двигателя, согласующий логические уровни микроконтроллера с силовой частью. Управление осуществляется через выходные линии порта, одна из которых задает режим работы драйвера, а вторая, подключенная к выходу таймера-счетчика, формирует сигнал широтно-импульсной модуляции для плавного регулирования оборотов двигателя.

Мониторинг параметров окружающей среды выполняется цифровым датчиком температуры и влажности. Обмен данными с микроконтроллером производится по однопроводному двунаправленному интерфейсу, подключенному к линии порта D, что позволяет считывать показания в цифровом виде без использования аналого-цифрового преобразования.

Канал связи с персональным компьютером организован на базе преобразователя интерфейсов USB-UART. Данный узел обеспечивает трансляцию сигналов шины USB в последовательный асинхронный протокол USART микроконтроллера, позволяя осуществлять дистанционный мониторинг и настройку системы.

1.5 Описание микроконтроллера ATmega8515

В качестве вычислительного ядра проектируемой системы выбран 8-битный микроконтроллер ATmega8515, построенный на базе архитектуры AVR RISC. Данная архитектура характеризуется разделением шин доступа к памяти программ и памяти данных, благодаря Гарвардской архитектуре, что обеспечивает возможность одновременной выборки инструкции и данных. Большинство команд выполняется за один машинный цикл, благодаря чему устройство достигает производительности до 16 MIPS при тактовой частоте 16 МГц [1].

Структурная организация микроконтроллера включает 32 рабочих регистра общего назначения, непосредственно связанных с арифметико-логическим устройством (АЛУ). Такая организация позволяет выполнять арифметические и логические операции между регистрами за один такт, что существенно повышает эффективность кода по сравнению с традиционными CISC-архитектурами.

Подсистема памяти микроконтроллера разделена на три независимых области. Первая область представляет собой перепрограммируемую Flash-память объемом 8 Кбайт, предназначенную для хранения программного кода. Вторая область — это оперативная память SRAM объемом 512 байт, используемая для хранения переменных и стека во время выполнения программы. Третья область — энергонезависимая память EEPROM объемом 512 байт. В рамках данного курсового проекта EEPROM используется для хранения настроек системы, таких как пороги температуры и режимы работы, что обеспечивает их сохранность при отключении питания.

Периферийные модули микроконтроллера включают два таймера-счетчика. Восьмибитный Таймер/Счетчик 0 в данной разработке сконфигурирован для работы в режиме генерации широтно-импульсной модуляции ШИМ, что позволяет управлять мощностью, подаваемой на электродвигатель вентилятора. Для организации обмена данными с персональным компьютером задействован универсальный асинхронный приемопередатчик UART, поддерживающий полнодуплексную связь.

Взаимодействие с внешними устройствами осуществляется через порты ввода-вывода. Микроконтроллер в корпусе PDIP-40 предоставляет 35 линий ввода-вывода, сгруппированных в пять портов. Расположение выводов корпуса приведено на рисунке 3.

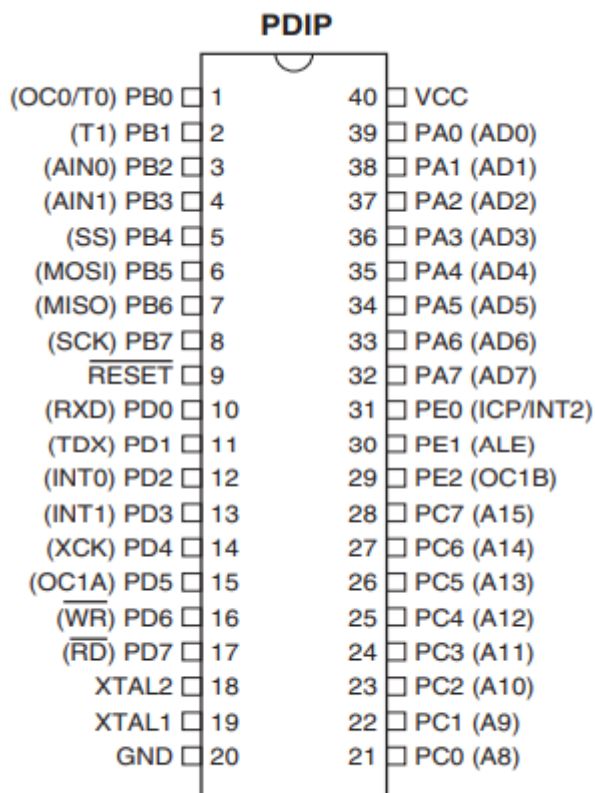


Рис. 3 — Расположение выводов микроконтроллера ATmega8515

1.6 Принцип работы и DRV8871

Управление двигателем постоянного тока требует согласования низковольтных логических сигналов микроконтроллера с силовыми цепями питания мотора. Эту функцию выполняет специализированная интегральная микросхема DRV8871.

Внутренняя структура драйвера базируется на топологии Н-моста, состоящего из четырех N-канальных полевых транзисторов (MOSFET). Применение полевых транзисторов вместо биполярных позволяет существенно снизить сопротивление открытого канала, которое составляет типовое значение 565 мОм [3]. Это техническое решение минимизирует тепловые потери и позволяет использовать драйвер без внешнего радиатора при рабочих токах двигателя вентилятора.

Функциональная схема драйвера, отображающая внутренние связи между логическим блоком и силовой частью, приведена на рисунке 4.

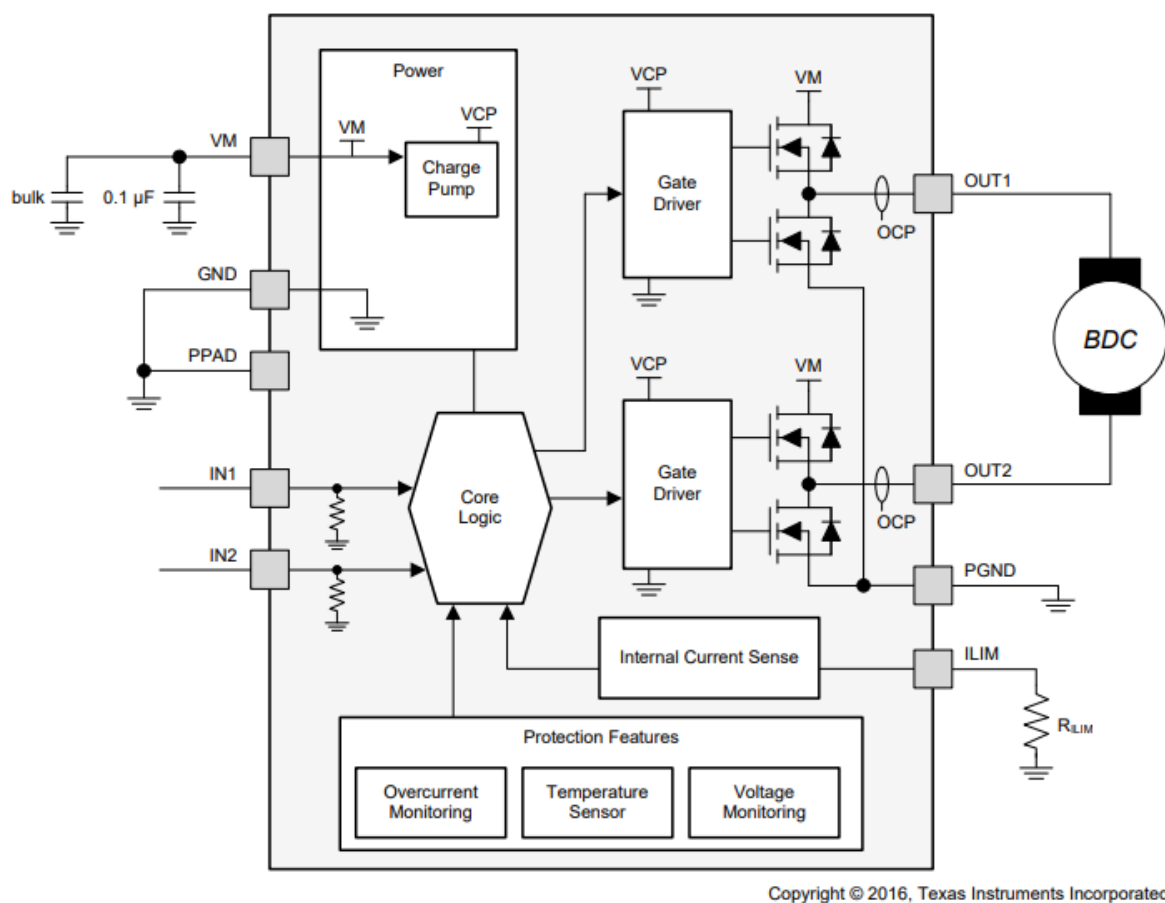


Рис. 4 — Функциональная схема драйвера DRV8871

Для управления верхними ключами Н-моста, выполненными на N-канальных транзисторах, микросхема содержит встроенный блок накачки заряда. Данный узел автоматически генерирует напряжение, превышающее напряжение питания двигателя, что необходимо для полного открытия верхних транзисторов.

Логика управления драйвером реализована через два входа IN1 и IN2. Согласно таблице истинности, приведенной в технической документации, комбинация логических уровней на этих входах определяет состояние выходов OUT1 и OUT2: вращение вперед, вращение назад, торможение или свободный выбег. Входы оснащены внутренними стягивающими резисторами, что предотвращает несанкционированное включение двигателя при высокоимпедансном состоянии выводов управляющего микроконтроллера.

Важным аспектом проектирования является анализ гальванической развязки. Изучение функциональной схемы и назначения выводов микросхемы DRV8871 показывает, что выводы земли логической части и силовой части электрически объединены внутри кристалла. Это свидетельствует об отсутствии гальванической развязки между управляющим микроконтроллером и силовой цепью. Безопасность низковольтной части обеспечивается высоким входным импедансом логических входов и внутренней архитектурой драйвера, где управляющие сигналы отделены от силовой шины через затворы транзисторов и схемы сдвига уровня. Дополнительную защиту обеспечивают встроенные системы блокировки при пониженном напряжении (UVLO) и защиты от сверхтоков (OCP), которые отключают силовые выходы при возникновении аварийных режимов.

Для реализации функции ограничения тока драйвер использует схему измерения падения напряжения на транзисторах. Порог срабатывания защиты задается единственным внешним резистором на выводе ILIM, что исключает необходимость применения мощных токоизмерительных шунтов в цепи двигателя.

1.7 Описание принципиальной электрической схемы

Принципиальная электрическая схема устройства разработана на основании функциональной схемы и определяет полный состав электронных компонентов и связей между ними. Схема изображена на рисунке 5.

Центральным элементом системы является микроконтроллер DD4 ATmega8515. Для обеспечения тактирования используется внешний кварцевый резонатор Z1, подключенный к выводам XTAL1 и XTAL2. Конденсаторы C9 и C10 номиналом 22 пФ обеспечивают необходимую нагрузочную емкость для стабильного возбуждения генератора. Цепь начального сброса реализована на резисторе R7 (10 кОм), который подтягивает вывод RESET к шине питания, предотвращая ложные срабатывания. Кнопка SA1, подключенная к выводу RESET через конденсатор, предназначена для ручного сброса микроконтроллера пользователем. Разъем XP3 предназначен для внутрисхемного программирования микроконтроллера.

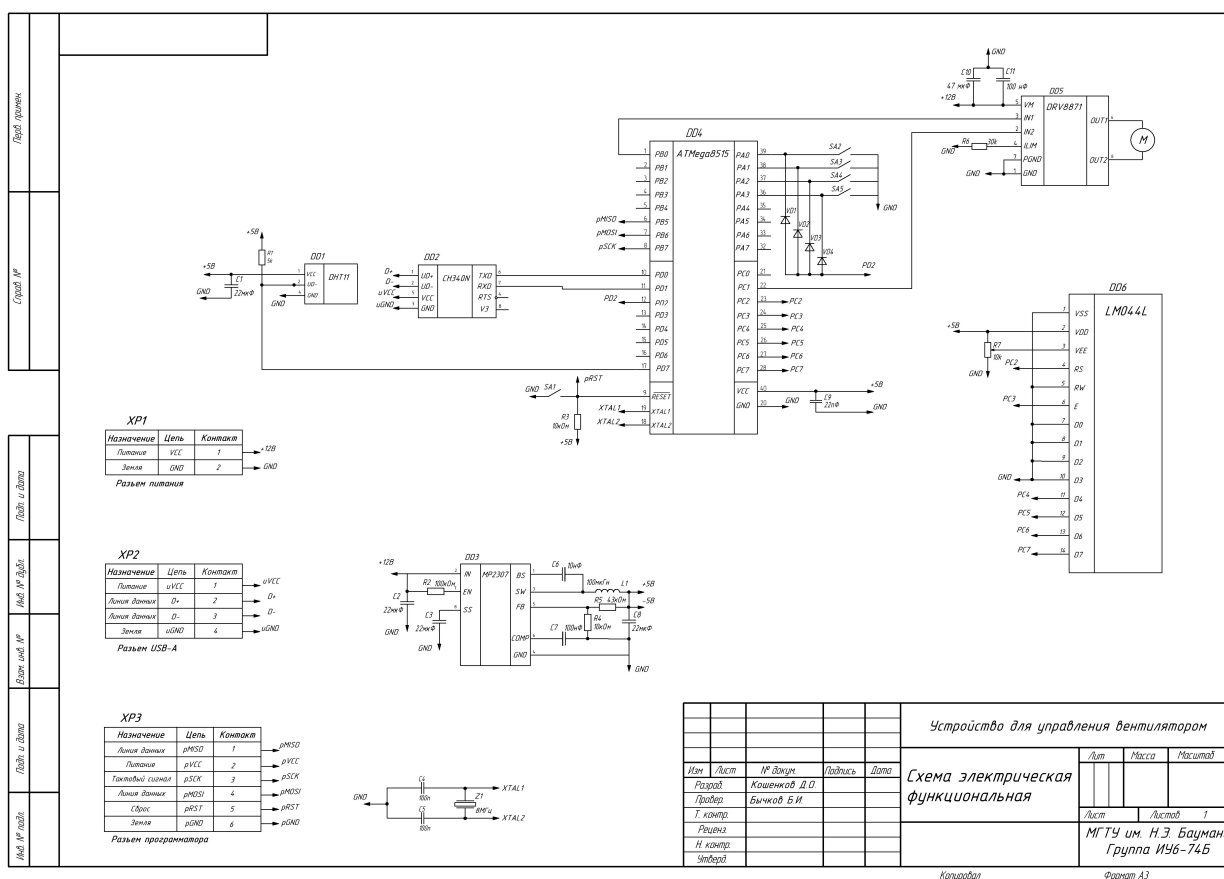


Рис. 5 — Схема электрическая потенциальная

Система питания устройства построена по импульсной схеме понижения напряжения. Входное напряжение +12 В поступает через разъем «Контакт 1». Для питания силовой части (драйвера двигателя) напряжение +12 В используется напрямую. Для питания логической части применена микросхема DD3 MP2307 — синхронный понижающий преобразователь напряжения. Резистивный делитель R5-R4 в цепи обратной связи, вывод FB, задает выходное напряжение на уровне 5 В. Резистор R2 подтягивает вход включения к питанию, обеспечивая автоматический запуск преобразователя.

Управление электродвигателем М осуществляется через специализированный драйвер DD5 DRV8871. Микросхема подключена к силовой шине +12 В, вывод VM. Конденсаторы C10 22 мкФ и C11 100 нФ выполняют функцию фильтрации питания драйвера, предотвращая просадки напряжения при коммутации обмоток двигателя. Резистор R6 30 кОм, подключенный к выводу ILM, задает порог ограничения тока двигателя на уровне 2.1 А, защищая его и источник питания от перегрузок. Управляющие сигналы поступают от

микроконтроллера на входы IN1 и IN2. Выходы OUT1 и OUT2 подключены непосредственно к клеммам двигателя постоянного тока.

Блок индикации реализован на жидкокристаллическом дисплее DD6 LM016L. Подключение выполнено по четырехбитной шине данных, D4-D7 дисплея подключены к выводам PC4-PC7 микроконтроллера. Управляющие сигналы RS и E подключены к выводам PC2 и PC3 соответственно. Вывод R/W дисплея соединен с землей, что переводит индикатор в режим постоянной записи. Подстроечный резистор R8 10 кОм образует делитель напряжения для регулировки контрастности изображения на дисплее.

Интерфейс ввода включает четыре тактовые кнопки SA2-SA5, подключенные к порту A микроконтроллера (PA0-PA3). Схемотехническое решение включает диодную развязку на элементах VD1-VD4. Аноды диодов подключены к линиям кнопок, а катоды объединены и заведены на линию внешнего прерывания PD2/INT0. Такое включение позволяет микроконтроллеру фиксировать факт нажатия любой из кнопок через генерацию прерывания, выводя устройство из режима энергосбережения, после чего программа определяет конкретную нажатую кнопку путем опроса порта A.

Взаимодействие с персональным компьютером организовано через интерфейс USB. Преобразование интерфейса USB в UART выполняет микросхема DD2 (CH340N). Линии D+ и D- подключены непосредственно к разъему USB. Линии приема (RXD) и передачи (TXD) соединены с соответствующими выводами порта D микроконтроллера (PD0 и PD1). Конденсатор C12 (100 нФ) служит для фильтрации внутреннего опорного напряжения микросхемы CH340N.

Датчик температуры и влажности DD1 (DHT11) подключен к шине питания +5 В. Информационный вывод датчика соединен с портом ввода-вывода микроконтроллера (PD7). Конденсатор C1 (22 мкФ) подключен между выводом питания датчика и землей для стабилизации питания и снижения помех. Резистор R1 (5.1 кОм) обеспечивает подтяжку информационной

линии датчика к высокому уровню, что является требованием спецификации протокола 1-Wire.

Представленная принципиальная схема обеспечивает реализацию всех функций, заложенных в техническом задании, включая автоматическое и ручное управление, индикацию, защиту силовых цепей и обмен данными с внешними устройствами.

1.8 Описание программной реализации и алгоритмов

1.8.1 Алгоритм основного цикла

После подачи питания происходит инициализация портов ввода-вывода, настройка UART, LCD-дисплея и внешних прерываний. Далее из энергонезависимой памяти EEPROM загружаются сохраненные настройки: режим работы, пороги температуры и скорость вентилятора.

В основном цикле последовательно выполняются следующие действия:

1) проверяется флаг готовности команды `cmd_ready`, и в случае его установки происходит разбор принятой строки с обновлением глобальных переменных и сохранением данных в EEPROM;

2) с помощью программного счетчика и функции задержки формируется интервал опроса датчика, по истечении которого вызывается функция чтения данных;

3) в автоматическом режиме устройство сравнивает текущую температуру с заданными порогами и реализует гистерезис, включая вентилятор при превышении верхнего порога и выключая его при падении ниже нижнего;

4) при изменении любых параметров или успешном чтении датчика выставляется флаг необходимости обновления дисплея, инициирующий вывод актуальной информации.

Схема алгоритма основного цикла представлена на рисунке 6:



Рисунок 6 — Схема алгоритма основного цикла

1.8.2 Алгоритм обработки обновления состояния вентилятора

Для автоматического управления охлаждением реализован алгоритм с гистерезисом, предотвращающий частые переключения двигателя при колебаниях температуры около порогового значения (см. рисунок 7).

Логика работы включает следующие этапы:

- 1) сравнивается текущее значение температуры с заданным верхним порогом, и, если температура превышает его при выключенном вентиляторе, производится включение двигателя;
- 2) в случае нахождения температуры ниже верхнего порога проверяется

условие нижнего порога;

3) если температура опустилась ниже заданного минимума и вентилятор находится в активном состоянии, формируется команда на его выключение, а в остальных случаях состояние системы остается без изменений.

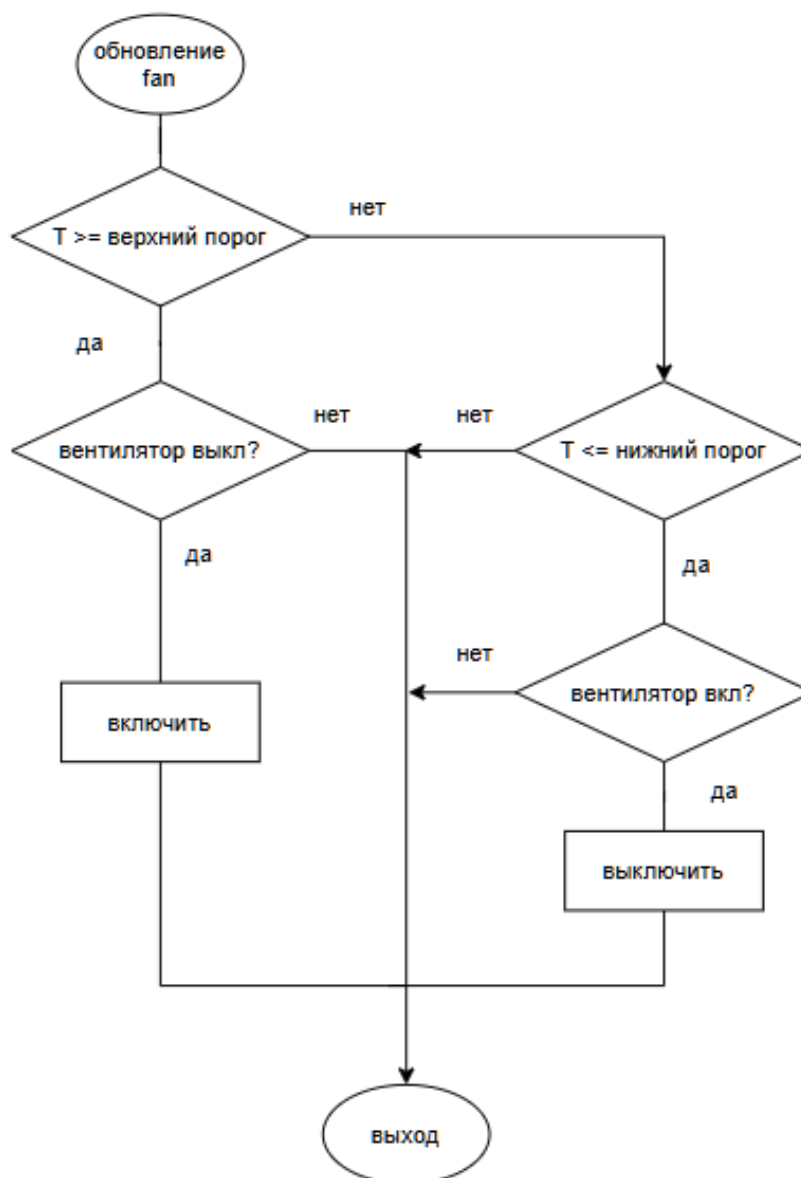


Рисунок 7 — Схема алгоритма обработки обновления состояния вентилятора

1.8.3 Алгоритм обработки команды UART

Обработка входящих данных производится по факту установки флага готовности команды, после чего анализируется содержимое приемного буфера.

Процедура разбора команд выполняется следующим образом:

- 1) проверяется первый символ буфера, определяющий тип запрашиваемого действия;
- 2) для команд ручного управления („1“ и „0“) система переводится в ручной режим, изменяется состояние мотора и обновляется запись в EEPROM;
- 3) при получении команд смены режима („A“ и „M“) устанавливается соответствующий флаг автоматического или ручного управления с сохранением настройки;
- 4) для команд с параметрами („V“, „H“, „L“) производится преобразование строкового аргумента в число, проверка на вхождение в допустимый диапазон (0–255 для скорости, 0–99 для температуры) и обновление соответствующих глобальных переменных и ячеек памяти;
- 5) по завершении операций флаг готовности команды сбрасывается для разрешения приема следующих данных.

Схема алгоритма обработки команды UART представлена на рисунке 8:

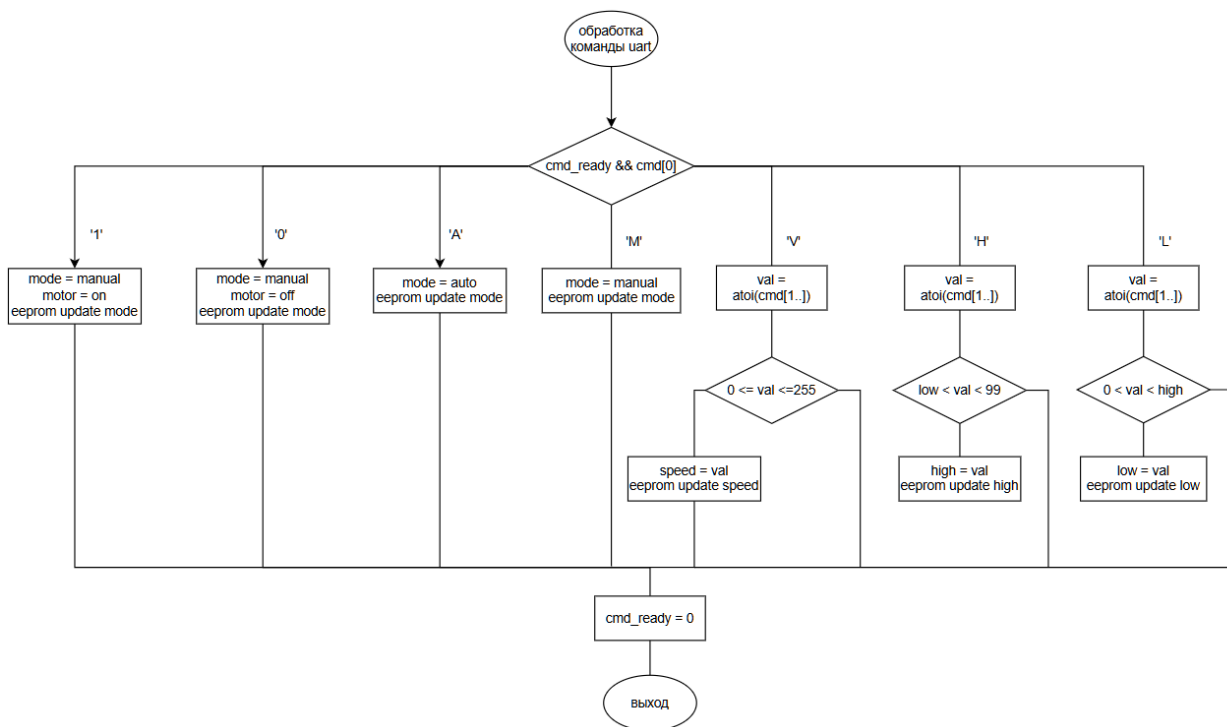


Рисунок 8 — Схема алгоритма обработки команды UART

1.9 Питание устройства

Для обеспечения функционирования устройства выбрана схема с единым входным напряжением +12 В, которое подается через разъем «Контакт 1». Данное напряжение необходимо для питания мощной нагрузки —

электродвигателя вентилятора. Однако микроконтроллер, дисплей, датчики и микросхема интерфейса требуют напряжения питания +5 В.

Для преобразования напряжения +12 В в +5 В применена микросхема DD3 — MP2307 [6]. Это монолитный синхронный понижающий импульсный стабилизатор напряжения. Выбор импульсного преобразователя вместо линейного обусловлен необходимостью минимизации тепловых потерь. При понижении 12 В до 5 В линейный стабилизатор рассеивал бы значительную мощность в виде тепла, что потребовало бы установки громоздкого радиатора. КПД микросхемы MP2307 достигает 95%, что обеспечивает высокую энергоэффективность устройства.

Схема включения MP2307 включает:

- входные конденсаторы C7 и C8 22 мкФ, объемный накопитель для фильтрации входного напряжения и стабилизации питания при переходных процессах;
- резистор R2, подтягивающий вход включения к питанию, обеспечивающий автоматический запуск преобразователя при подаче питания;
- дроссель индуктивностью 10 мкГн и выходной конденсатор C5 22 мкФ, образующие LC-фильтр для сглаживания пульсаций выходного напряжения;
- делитель напряжения R5-R4, 43 кОм и 10 кОм соответственно в цепи обратной связи, задающий выходное напряжение стабилизатора на уровне 5.0 В;
- конденсатор компенсации C6 10 нФ между выводом COMP и GND, обеспечивающий формирование полюса цепи обратной связи и устойчивость системы.

Таким образом, блок питания обеспечивает стабильное напряжение для цифровой части схемы и высокий ток для силовой части от одного источника.

1.10 Интерфейс программирования

Для загрузки программного обеспечения во Flash-память микроконтроллера предусмотрен разъем XP3. Программирование осуществляется по последовательному интерфейсу ISP (In-System Programming). Разъем подключен к следующим выводам микроконтроллера:

- MOSI (PB5), вход данных для последовательного программирования;
- MISO (PB6), выход данных;
- SCK (PB7), тактовый сигнал интерфейса SPI;
- RESET (Pin 9), для перевода микроконтроллера в режим программирования программатор подает на этот вывод логический ноль;
- VCC и GND, линии питания для согласования уровней сигналов программатора и целевого устройства.

1.11 Расчетная часть

В данном разделе приведены расчеты пассивных компонентов обвязки микросхем, параметров программного обеспечения и энергетического баланса устройства.

1.11.1 Расчет элементов драйвера двигателя (DRV8871)

Микросхема DRV8871 [3] оснащена функцией регулирования тока, которая ограничивает ток через нагрузку на заданном уровне. Это необходимо для защиты двигателя при заклинивании и защиты источника питания от пусковых токов.

Порог ограничения тока I_{TRIP} задается резистором R_6 , подключенным к выводу ILIM. Согласно технической документации (раздел 7.3.3 даташита), зависимость тока от сопротивления описывается формулой:

$$I_{TRIP} = \frac{V_{ILIM}}{R_6} \quad (1)$$

где: – $V_{ILIM} = 64$ мВ — типовое значение опорного напряжения на выводе ILIM (диапазон 59–69 мВ);

- R_6 — сопротивление резистора в Омах;
- I_{TRIP} — ток срабатывания защиты в Амперах.

В более удобной для практического расчета форме, если R_6 выражен в кОм, а ток в Амперах:

$$R_6[\text{кОм}] = \frac{0.064}{I_{TRIP}}[\text{A}] \quad (2)$$

Для используемого в проекте вентилятора типовой рабочий ток составляет 0.2–0.3 А, пусковой ток может достигать 1–1.5 А. Максимальный ток драйвера составляет 3.6 А. Для обеспечения безопасной работы и запаса по мощности установим порог ограничения тока на уровне $I_{\text{TRIP}} \approx 2.1\text{А}$.

Рассчитаем номинал резистора:

$$R_6 = 0.064 \frac{\text{В}}{2.1} \text{А} \approx 0.0305 \text{ Ом} \approx 30.5 \text{ мОм} \quad (3)$$

или в кОм:

$$R_6 = \frac{0.064}{2.1} \approx 30.47 \text{ кОм} \quad (4)$$

Выбираем ближайшее стандартное значение из ряда E24: $R_6 = 30 \text{ кОм}$ (элемент R7 на принципиальной схеме).

При таком сопротивлении фактический ток ограничения составит:

$$I_{\text{TRIP}} = 0.064 \frac{\text{В}}{30} \text{ кОм} \approx 2.13\text{А} \quad (5)$$

Такое значение защитит драйвер от короткого замыкания и перегрузки по току, но не будет ограничивать ток при нормальном старте вентилятора (1–1.5 А).

Подсистема питания драйвера реализована с использованием двух конденсаторов. Высокочастотный керамический конденсатор C11 (100 нФ, X7R) подключен как можно ближе к выводам VM и GND микросхемы и подавляет высокочастотные помехи при коммутации ключей. Объемный электролитический конденсатор C10 (22 мкФ, напряжение не ниже 16 В) стабилизирует напряжение питания при резких изменениях тока двигателя. Дополнительно между выводами SW и BS установлен bootstrap-конденсатор

(100 нФ керамический), обеспечивающий питание внутреннего драйвера верхнего плеча H-моста.

Для питания цифровой части схемы используется понижающий DC-DC преобразователь MP2307 [6], преобразующий входные 12 В в стабилизированные 5 В.

Выходное напряжение V_{OUT} задается резистивным делителем, подключенным к выводу FB (Feedback). опорное напряжение микросхемы $V_{FB} = 0.925\text{В}$ (типовое значение, диапазон 0.900–0.950 В). Формула для расчета:

$$V_{OUT} = V_{FB} \times \left(1 + \frac{R_{upper}}{R_{lower}} \right) \quad (6)$$

Согласно принципиальной схеме, нижний резистор делителя имеет номинал $R4 = 10 \text{ кОм}$. Определим необходимое значение верхнего резистора $R5$ для получения $V_{OUT} = 5.0\text{В}$:

$$R5 = R4 \times \left(\frac{V_{OUT}}{V_{FB}} - 1 \right) = 10 \times \left(\frac{5.0}{0.925} - 1 \right) = 10 \times (5.405 - 1) \approx 44.05 \text{ кОм}$$

Выбираем ближайшее стандартное значение из ряда E24: $R5 = 43 \text{ кОм}$.

Проверим фактическое выходное напряжение:

$$V_{OUT} = 0.925 \times \left(1 + \frac{43}{10} \right) = 0.925 \times 5.3 = 4.9025\text{В} \quad (8)$$

Полученное значение 4.90 В находится в пределах допустимого диапазона питания микроконтроллера ATmega8515 (4.5–5.5 В). На принципиальной схеме верхний резистор делителя соответствует элементу $R5$, нижний — $R4$.

Согласно рекомендациям таблицы в даташите для выходного напряжения 5 В оптимальной является индуктивность $L = 10 \text{ мкГн}$. Такой

номинал обеспечивает быстрый отклик на изменение нагрузки, компактные габариты дросселя и достаточный запас по допустимому току.

Выходной конденсатор C5 определяет величину пульсации напряжения на выходе преобразователя. Для керамического конденсатора с низким ESR (тип X5R/X7R) даташит рекомендует емкость не менее 22 мкФ.

Пульсация выходного напряжения:

$$\Delta V_{\text{OUT}} = \left(\Delta \frac{I_L}{8 \times f_{\text{SW}} \times C} \right) + \Delta V_{\text{ESR}} \quad (9)$$

Примем $\Delta I_L = 0.3\text{A}$, $f_{\text{SW}} = 340\text{ кГц}$, $C5 = 22\text{ мкФ}$, $\text{ESR} \approx 0.05\text{ Ом}$:

$$\begin{aligned} \Delta V_{\text{OUT}} &= \frac{0.3}{8 \times 340000 \times 22 \times 10^{-6}} + 0.3 \times 0.05 \approx \\ &\approx 50\text{ мВ} + 15\text{ мВ} \approx 65\text{ мВ} \end{aligned} \quad (10)$$

Полученная пульсация значительно меньше допустимого отклонения питания микроконтроллера ($\pm 5\%$ от 5 В, т.е. $\pm 250\text{ мВ}$). Таким образом, выбор $C5 = 22\text{ мкФ}$ керамический X5R/X7R (напряжение не ниже 10 В, $\text{ESR} < 100\text{ мОм}$) обеспечивает стабильность выходного напряжения.

Для стабильной работы преобразователя необходимы два входных конденсатора:

- $C7 = 100\text{ нФ}$ — керамический конденсатор X7R, подключенный максимально близко к выводам IN и GND микросхемы, для подавления высокочастотных помех;

- $C8 = 22\text{ мкФ}$ — электролитический или керамический конденсатор (напряжение не ниже 16 В) для компенсации провалов питания при переходных процессах.

РМС-ток через входной объемный конденсатор:

$$\begin{aligned}
I_{\text{RMS}} &= I_{\text{LOAD}} \times \sqrt{\left(\frac{V_{\text{IN}}}{V_{\text{OUT}}} - 1\right) \times \left(\frac{V_{\text{OUT}}}{V_{\text{IN}}}\right)} = \\
&= 0.5 \times \sqrt{\left(\frac{12}{5} - 1\right) \times \left(\frac{5}{12}\right)} \approx 0.5\text{A}
\end{aligned} \tag{11}$$

Выбранный номинал $C8 = 22 \text{ мкФ}$ с низким ESR обеспечивает достаточный запас по допустимому РМС-току.

Конденсатор компенсации $C6$ между выводом COMP и GND формирует полюс цепи обратной связи и обеспечивает устойчивость системы. Согласно рекомендациям даташита, при использовании керамического выходного конденсатора с низким ESR рекомендуется C_{COMP} в диапазоне 3.9–10 нФ. В проекте выбран $C6 = 10 \text{ нФ}$ керамический X7R, что обеспечивает запас по устойчивости и приемлемую частоту среза контура регулирования. Номинал более 10 нФ был бы избыточным и мог бы ухудшить динамику, поэтому он установлен на рекомендованном значении.

Резистор $R2 = 100 \text{ кОм}$ подтягивает вход EN (Enable) к шине питания, обеспечивая автоматический запуск преобразователя при подаче питания. Такой номинал соответствует рекомендациям производителя и обеспечивает малый дополнительный ток утечки.

Тактовая частота микроконтроллера задается внешним кварцевым резонатором: $f_{\text{osc}} = 8 \text{ МГц}$. Конденсаторы $C9$ и $C10$ номиналом 22 пФ подключены между выводами кварцевого резонатора и землей, обеспечивая необходимую нагрузочную емкость для стабильного возбуждения генератора.

Для связи с персональным компьютером выбрана скорость обмена $\text{Baud} = 9600 \text{ бит/с}$ — стандартное значение для асинхронного последовательного интерфейса.

Значение регистра UBRR (USART Baud Rate Register) рассчитывается по формуле:

$$\text{UBRR} = \frac{f_{\text{osc}}}{16 \times \text{Baud}} - 1 \tag{12}$$

$$\text{UBRR} = \frac{8 \times 10^6}{16 \times 9600} - 1 = 52.083 - 1 \approx 51 \quad (13)$$

Рассчитаем реальную скорость при $\text{UBRR} = 51$:

$$\text{Baud}_{\text{real}} = \frac{f_{\text{osc}}}{16 \times (\text{UBRR} + 1)} = \frac{8000000}{16 \times 52} \approx 9615 \text{ бит/с} \quad (14)$$

Погрешность скорости:

$$\begin{aligned} \text{Error} &= \left| \frac{\text{Baud}_{\text{real}} - \text{Baud}_{\text{nominal}}}{\text{Baud}_{\text{nominal}}} \right| \times 100\% = \\ &= \left| \frac{9615 - 9600}{9600} \right| \times 100\% \approx 0.16\% \end{aligned} \quad (15)$$

Погрешность менее 0.2 % является отличным показателем и гарантирует стабильную передачу данных.

Для управления скоростью вентилятора используется 8-битный Таймер/Счетчик 0 в режиме Fast PWM. Предделитель тактовой частоты установлен на $N = 8$.

Частота ШИМ-сигнала:

$$f_{\text{PWM}} = \frac{f_{\text{osc}}}{N \times 256} = \frac{8 \times 10^6}{8 \times 256} = 3906.25 \text{ Гц} \approx 3.9 \text{ кГц} \quad (16)$$

Частота около 3.9 кГц является компромиссной: – достаточно высока для обеспечения плавного вращения двигателя благодаря его индуктивности;

– не приводит к чрезмерным потерям на переключение в драйвере DRV8871;

– лежит выше диапазона частот, на которых наиболее заметно

механическое дребезжание, хотя и ниже верхней границы слышимого диапазона.

1.11.2 Расчет энергопотребления и выбор источника питания

Расчет производится для режима максимальной нагрузки при напряжениях питания +5 В (логическая часть) и +12 В (силовая часть).

Элементы схемы:

- микроконтроллер ATmega8515 (активный режим, 8 МГц): $I_{MCU} \approx 12$ мА;
- Дисплей LM016L (подсветка + контроллер): $I_{LCD} \approx 22$ мА (около 20 мА на подсветку и 2 мА на контроллер);
- датчик DHT11 (режим измерения): $I_{DHT} \approx 2.5$ мА;
- преобразователь CH340N: $I_{USB} \approx 15$ мА.

Суммарный ток:

$$I_{5V} = 12 + 22 + 2.5 + 15 = 51.5 \text{ мА} \quad (17)$$

Мощность логической части:

$$P_{LOGIC} = 5V \times 0.0515A \approx 0.26 \text{ Вт} \quad (18)$$

1.11.3 Потребление по цепи +12 В

Основным потребителем является вентилятор. Для типового корпусного вентилятора 12 В рабочий ток составляет 0.2–0.3 А. Примем $I_{FAN} = 0.3A$.

Мощность вентилятора:

$$P_{FAN} = 12V \times 0.3A = 3.6 \text{ Вт} \quad (19)$$

Преобразователь MP2307 имеет КПД $\eta \approx 90\%$ при нагрузке около 0.5 А. Мощность, потребляемая им от шины 12 В:

$$P_{IN_DC} = \frac{P_{LOGIC}}{\eta} = \frac{0.26}{0.9} \approx 0.29 \text{ Вт} \quad (20)$$

Ток, потребляемый преобразователем от 12 В:

$$I_{DC_IN} = 0.29 \frac{\text{Вт}}{12 \text{ В}} \approx 0.024 \text{ А} = 24 \text{ мА} \quad (21)$$

1.11.4 Общее потребление устройства

Суммарный ток от источника 12 В:

$$I_{TOTAL} = I_{FAN} + I_{DC_IN} = 0.3 + 0.024 = 0.324 \text{ А} \approx 324 \text{ мА} \quad (22)$$

Суммарная потребляемая мощность:

$$P_{TOTAL} = 12 \text{ В} \times 0.324 \text{ А} \approx 3.9 \text{ Вт} \quad (23)$$

1.11.5 Выбор источника питания

С учетом пусковых токов вентилятора (до 1–1.5 А кратковременно) целесообразно выбрать источник питания с запасом по току. Требуемые характеристики источника:

- выходное напряжение: 12 В ($\pm 5\%$);
- номинальный ток: не менее 0.5 А;
- допустимый кратковременный ток: до 1 А.

Практически удобно использовать стандартный импульсный блок питания 12 В / 1 А, обеспечивающий необходимый запас по мощности и устойчивую работу устройства.

2 Технологическая часть

2.1 Работа в системах разработки и отладки

Для реализации проекта был выбран комплекс современных программных средств, позволяющий осуществлять сквозное проектирование от разработки алгоритма до прошивки кристалла. Написание и компиляция управляющей программы производились в интегрированной среде AVR Studio. В качестве компилятора использовался AVR-GCC с языком C, что позволило существенно ускорить процесс разработки и повысить читаемость кода по сравнению с использованием языка Ассемблера. Менеджер проектов данной среды предоставляет инструменты для конфигурирования параметров микроконтроллера, таких как тип кристалла, тактовая частота и уровни оптимизации компилятора.

Для программной реализации алгоритмов были использованы внешние библиотеки. Библиотека `avr/io.h` предоставляет определения имен регистров ввода-вывода и битов микроконтроллера ATmega8515, используемых для управления портами и периферией. Библиотека `avr/interrupt.h` предоставляет макрос `ISR` для объявления функций-обработчиков прерываний, а также функции глобального управления прерываниями. Для реализации временных задержек, необходимых для инициализации дисплея и формирования таймингов протокола 1-Wire датчика DHT11, применена библиотека `util/delay.h`.

Особенностью данного проекта является активное использование энергонезависимой памяти и строковых констант. Библиотека `avr/eeprom.h` используется для чтения и сохранения настроек пользователя при отключении питания. Для экономии оперативной памяти, объем которой составляет всего 512 байт, строковые литералы размещены во Flash-памяти программ с помощью макроса `PSTR` из библиотеки `avr/pgmspace.h`. Преобразование строковых команд, поступающих по интерфейсу UART, в числовые значения выполняется функциями стандартной библиотеки `stdlib.h`.

Для проверки аппаратной части без необходимости физической пайки применялся пакет схемотехнического моделирования Proteus 8 Professional.

Ключевой особенностью данной системы является возможность совместной симуляции аналоговых и цифровых компонентов вместе с исполняемым кодом микроконтроллера в реальном времени. В проекте использовались математические модели микроконтроллера ATmega8515, драйвера двигателя DRV8871, символьного дисплея LM016L, датчика DHT11 и виртуального терминала для отладки протокола UART. Симуляция устройства в среде Proteus представлена на рисунке 9

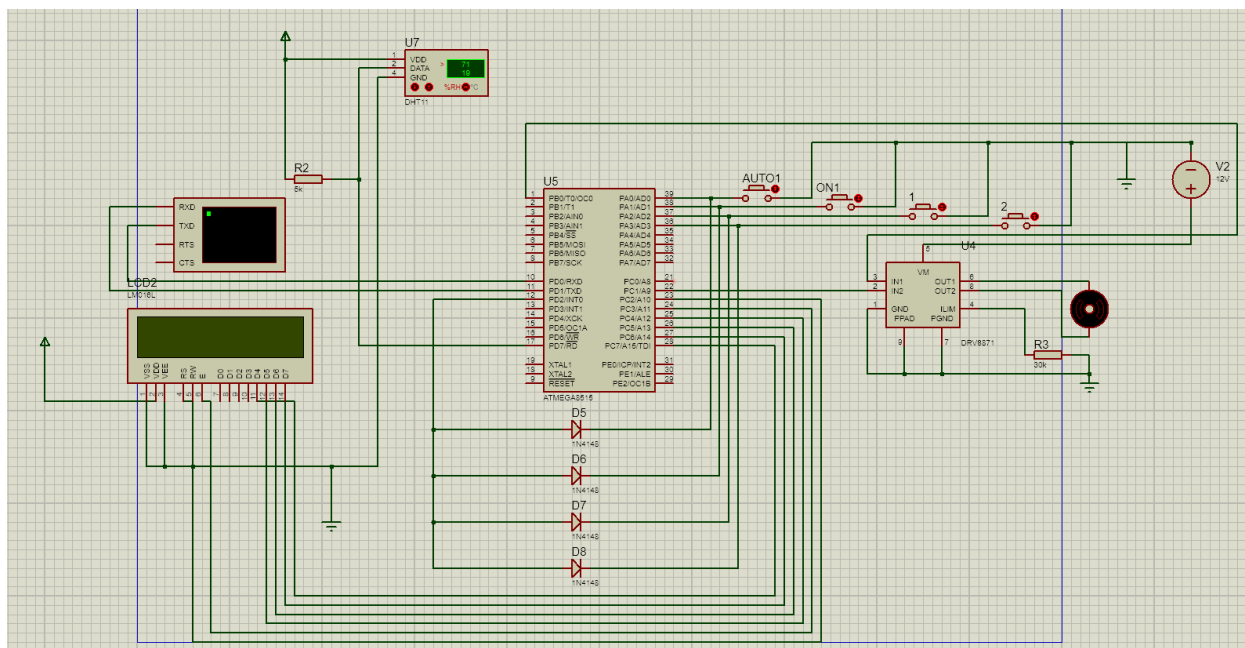


Рисунок 9 — Моделирование в Proteus

2.2 Структура программного обеспечения

Программное обеспечение микроконтроллера разработано на языке высокого уровня C. Архитектура приложения построена по принципу суперцикла с использованием прерываний для обработки асинхронных событий. Программа логически разделена на модуль инициализации, выполняющий настройку периферии при старте, модули драйверов для работы с LCD и датчиком, модуль управления памятью и основной цикл. В основном цикле происходит опрос флагов, реализация автомата состояний термоконтроля и логики управления вентилятором. Обработка критичных ко времени событий, таких как прием данных по UART и реакция на нажатия кнопок, вынесена в обработчики прерываний.

2.3 Настройка периферийных устройств

Конфигурация направлений линий портов выполняется через регистры DDRx. К выходам отнесены линии управления двигателем, линии данных дисплея и ШИМ, а также линия датчика при отправке запроса. Входами являются линии кнопок, для которых программно активированы внутренние подтягивающие резисторы для обеспечения стабильного логического уровня в ненажатом состоянии.

Для управления скоростью вращения вентилятора используется 8-битный таймер-счетчик T0 в режиме быстрого ШИМ. Настройка производится через регистр TCCR0 путем установки битов, включающих режим Fast PWM, неинвертирующий режим выхода на выводе OC0 и делитель тактовой частоты. Это обеспечивает генерацию сигнала с частотой, оптимальной для управления драйвером двигателя.

Модуль UART настроен на скорость 9600 бод, что является стандартом для подобных систем. В регистрах управления разрешены прием и передача данных, а также прерывание по завершению приема, что позволяет обрабатывать входящие команды в фоновом режиме, не блокируя выполнение основной программы.

2.4 Макетная отладка и программирование микроконтроллера

Для загрузки управляющей программы в микроконтроллер используется метод внутрисхемного программирования (ISP) через интерфейс SPI. Для этого задействуются выводы MOSI, MISO, SCK и RESET, выведенные на разъем XP3.

Важным этапом является конфигурация Fuse-битов микроконтроллера. Для корректной работы устройства с внешним кварцевым резонатором на 8 МГц и сохранения данных в EEPROM при перепрошивке необходимо выставить значения, приведенные в таблице 6.

Таблица 6 — Конфигурация Fuse-битов для ATmega8515

Функция	Бит	Значение
---------	-----	----------

Продолжение таблицы 6

Выбор источника тактирования	CKSEL3..0	1111 (Ext. Crystal High Freq)
Время старта	SUT1..0	11 (Slow rising power)
Разрешение SPI	SPIEN	0 (Включено)
Сторожевой таймер	WDTON	1 (Выключено)
Сохранение EEPROM	EESAVE	0 (Включено)

2.5 Отладка и тестирование

2.6 Методика тестирования и проверочные тесты

Для подтверждения работоспособности разработанного программного обеспечения проведено тестирование функциональных модулей системы. Тестирование включало проверку реакции устройства на нажатие кнопок управления, прием команд по интерфейсу UART и автоматическую работу системы термоконтроля. Проверки выполнялись в среде симуляции Proteus.

Результаты тестирования приведены в таблице 7.

Таблица 7 — Таблица тестовых случаев

Входные данные	Ожидаемый результат	Фактический результат
Нажатие кнопки «AUTO» (PA0)	Переход в автоматический режим. Отправка «INT: AUTO Mode» по UART	Система перешла в режим AUTO. Сообщение получено
Нажатие кнопки «TOGGLE» (PA1)	Переключение в ручной режим. Смена состояния мотора. Отправка «INT: Toggle Man»	Режим изменен на MANUAL. Состояние мотора изменилось
Нажатие кнопки «UP» (PA2)	Увеличение скорости на 25. Обновление OCR0.	Скорость увеличилась. ШИМ изменился

Продолжение таблицы 7

	Отправка «INT: Speed UP: «	
Команда «А» по UART	Переход в автоматический режим. Ответ «Mode: AUTO»	Режим AUTO. Ответ корректен
Команда «1» по UART	Включение мотора в ручном режиме. Ответ «Manual ON»	Мотор включен. Ответ получен
Команда «V150» по UART	Установка скорости 150. Обновление ШИМ. Сохранение в EEPROM	Скорость установлена 150. ШИМ соответствует
Команда «H30» по UART	Установка верхнего порога 30°C. Ответ «High Saved: 30»	Порог изменен. Ответ получен
Режим AUTO. Температура 24°C, Порог H=23°C	Включение вентилятора	Вентилятор включился
Режим AUTO. Температура 20°C, Порог L=21°C	Выключение вентилятора	Вентилятор выключился

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было спроектировано, программно реализовано и протестировано микропроцессорное устройство для автоматизированного управления системой охлаждения. Поставленная цель работы достигнута, а требования технического задания выполнены в полном объеме.

В процессе проектирования получены следующие основные результаты:

1) обоснован выбор элементной базы и разработана схема электрическая принципиальная, обеспечивающая корректное согласование микроконтроллера ATmega8515 с цифровым датчиком DHT11 и силовым драйвером DRV8871;

2) реализован программный комплекс на языке C, включающий драйверы периферийных устройств, модуль обработки нестандартного однопроводного протокола и логику автоматического управления с гистерезисом;

3) организован развитый пользовательский интерфейс, сочетающий локальную индикацию параметров на символьном ЖК-дисплее и возможность удаленной настройки через последовательный интерфейс UART;

4) внедрена подсистема работы с энергонезависимой памятью (EEPROM), обеспечивающая сохранение уставок температуры и режима работы при отключении питания;

5) проведено тестирование и отладка устройства, подтвердившие корректность формирования временных интервалов и стабильность работы во всех режимах.

Занимаемый объем программного кода составляет менее 4 КБ Flash-памяти, что оставляет значительный запас ресурсов для возможной модернизации. Точность поддержания микроклимата определяется характеристиками датчика (± 2 °C), что является достаточным для задач управления бытовой вентиляцией.

В качестве направлений для дальнейшего совершенствования устройства можно рекомендовать внедрение ПИД-регулятора для более плавного изменения оборотов двигателя, а также добавление обратной связи по

тахометрическому датчику для контроля реальной скорости вращения вентилятора.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ATmega8515 8-bit Microcontroller with 8K Bytes In-System Programmable Flash. Datasheet [Электронный ресурс]. URL: <https://ww1.microchip.com/downloads/en/DeviceDoc/doc2512.pdf> (дата обращения: 20.05.2025).
2. DHT11 Humidity & Temperature Sensor. Datasheet [Электронный ресурс]. URL: <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf> (дата обращения: 20.05.2025).
3. DRV8871 3.6-V to 40-V Brushed DC Motor Driver with Internal Current Sensing. Datasheet [Электронный ресурс]. URL: <https://www.ti.com/lit/ds/symlink/drv8871.pdf> (дата обращения: 20.05.2025).
4. LM016L Liquid Crystal Display Module. Datasheet [Электронный ресурс]. URL: <https://www.hitachi-displays-eu.com/doc/LM016L.pdf> (дата обращения: 20.05.2025).
5. CH340 USB to Serial Chip. Datasheet [Электронный ресурс]. URL: https://www.wch-ic.com/downloads/CH340DS1_PDF.html (дата обращения: 20.05.2025).
6. MP2307 3A, 23V, 340KHz Synchronous Rectified Step-Down Converter. Datasheet [Электронный ресурс]. URL: https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MP2307/ (дата обращения: 20.05.2025).
7. ГОСТ 2.702-2011. Единая система конструкторской документации. Правила выполнения электрических схем. 2011.
8. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. 1990.
9. AVR Libc Reference Manual [Электронный ресурс]. URL: <https://www.nongnu.org/avr-libc/user-manual/> (дата обращения: 20.05.2025).

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ ТЕКСТ ПРОГРАММЫ
Листов 10

Листинг А.1 — Содержимое файла main.c

```
#define F_CPU 8000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/eeprom.h>
#include <avr/pgmspace.h>
#include <util/delay.h>
#include <stdlib.h>
#include <string.h>

// --- НАСТРОЙКИ ПИНОВ ---

// Моторы (PC0, PC1)
#define MOTOR_PORT PORTC
#define MOTOR_DDR DDRC
#define IN1_PIN PC0
#define IN2_PIN PC1

// ШИМ (PB0)
#define PWM_PORT PORTB
#define PWM_DDR DDRB
#define PWM_PIN PB0

// Датчик DHT (PD7)
#define DHT_PORT PORTD
#define DHT_DDR DDRD
#define DHT_PIN PD7
#define DHT_PIN_IN PIND

// КНОПКИ (PORTA)
// ВАЖНО: Для работы INT0, эти кнопки должны быть также
// электрически связаны с PD2 через диоды!
#define BTN_PORT PORTA
#define BTN_PIN PINA
#define BTN_DDR DDRA
#define BTN_AUTO PA0
#define BTN_TOGGLE PA1
#define BTN_UP PA2
#define BTN_DOWN PA3

// Пин прерывания (для информации)
#define INT0_PIN PD2

// LCD ЭКРАН (PC2-PC7)
#define LCD_PORT PORTC
#define LCD_DDR DDRC
```

Продолжение листинга А.1

```
#define LCD_RS      PC2
#define LCD_E       PC3

// -----

// eeprom
#define EE_MAGIC_VAL 0xAC

uint8_t EEMEM ee_magic;
uint8_t EEMEM ee_mode_auto;
uint8_t EEMEM ee_fan_on;
uint8_t EEMEM ee_temp_high;
uint8_t EEMEM ee_temp_low;
uint8_t EEMEM ee_fan_speed;

// переменные
volatile uint8_t fan_on = 0;
volatile uint8_t mode_auto = 1;
volatile uint8_t temp_c = 0;
volatile uint8_t humidity = 0;

volatile uint8_t temp_high = 23;
volatile uint8_t temp_low = 21;
volatile uint8_t fan_speed = 255;

// Буфер UART
volatile char rx_buffer[20];
volatile uint8_t rx_pos = 0;
volatile uint8_t cmd_ready = 0;

volatile uint8_t update_lcd_needed = 1;

// --- ФУНКЦИИ LCD ---

void lcd_pulse(void) {
    LCD_PORT |= (1 << LCD_E);
    _delay_us(5);
    LCD_PORT &= ~(1 << LCD_E);
    _delay_us(50);
}

void lcd_byte(uint8_t val, uint8_t is_data) {
    if (is_data) LCD_PORT |= (1 << LCD_RS);
    else         LCD_PORT &= ~(1 << LCD_RS);

    uint8_t temp = (LCD_PORT & 0x0F);
```


Продолжение листинга А.1

```
    temp |= (val & 0xF0);
    LCD_PORT = temp;
    lcd_pulse();

    temp = (LCD_PORT & 0x0F);
    temp |= ((val << 4) & 0xF0);
    LCD_PORT = temp;
    lcd_pulse();
}

void lcd_cmd(uint8_t cmd) {
    lcd_byte(cmd, 0);
    if (cmd == 0x01 || cmd == 0x02) _delay_ms(2);
}

void lcd_char(char data) { lcd_byte(data, 1); }

void lcd_str_P(const char *str) {
    while (pgm_read_byte(str)) lcd_char(pgm_read_byte(str++));
}

void lcd_str(const char *str) { while (*str) lcd_char(*str++); }

void lcd_num(int val) {
    char buf[10];
    itoa(val, buf, 10);
    lcd_str(buf);
}

void lcd_init(void) {
    LCD_DDR |= (1 << LCD_RS) | (1 << LCD_E) | 0xF0;
    _delay_ms(20);
    LCD_PORT &= ~(1 << LCD_RS);

    LCD_PORT = (LCD_PORT & 0x0F) | 0x30; lcd_pulse();
    _delay_ms(5);
    LCD_PORT = (LCD_PORT & 0x0F) | 0x30; lcd_pulse();
    _delay_us(200);
    LCD_PORT = (LCD_PORT & 0x0F) | 0x30; lcd_pulse();
    _delay_us(200);
    LCD_PORT = (LCD_PORT & 0x0F) | 0x20; lcd_pulse();
    _delay_ms(5);

    lcd_cmd(0x28);
    lcd_cmd(0x0C);
```

Продолжение листинга А.1

```
    lcd_cmd(0x06);
    lcd_cmd(0x01);
}

void lcd_gotoxy(uint8_t x, uint8_t y) {
    uint8_t addr = (y == 0) ? 0x80 : 0xC0;
    lcd_cmd(addr + x);
}

// --- КОНФИГУРАЦИЯ ---

void load_config(void) {
    uint8_t magic = eeprom_read_byte(&ee_magic);
    if (magic != EE_MAGIC_VAL) {
        eeprom_update_byte(&ee_mode_auto, 1);
        eeprom_update_byte(&ee_fan_on, 0);
        eeprom_update_byte(&ee_temp_high, 23);
        eeprom_update_byte(&ee_temp_low, 21);
        eeprom_update_byte(&ee_fan_speed, 255);
        eeprom_update_byte(&ee_magic, EE_MAGIC_VAL);
    }
    mode_auto = eeprom_read_byte(&ee_mode_auto);
    fan_on = eeprom_read_byte(&ee_fan_on);
    temp_high = eeprom_read_byte(&ee_temp_high);
    temp_low = eeprom_read_byte(&ee_temp_low);
    fan_speed = eeprom_read_byte(&ee_fan_speed);
}

// --- UART ---

void uart_init_fixed(void) {
    UBRRH = 0; UBRRL = 51; // 9600
    UCSRB = (1 << RXCIE) | (1 << RXEN) | (1 << TXEN);
    UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0);
}

void uart_send_char(char c) { while (!(UCSRA & (1 << UDRE)));
    UDR = c; }
void uart_send_str(const char *s) { while (*s)
    uart_send_char(*s++); }
void uart_send_str_P(const char *s) {
    while (pgm_read_byte(s)) uart_send_char(pgm_read_byte(s+
+));
}
void uart_send_num(int val) {
    char buf[10];
```

Продолжение листинга A.1

```
    itoa(val, buf, 10);
    uart_send_str(buf);
}

ISR(USART_RX_vect) {
    char data = UDR;
    if (data == '\r' || data == '\n') {
        rx_buffer[rx_pos] = 0; cmd_ready = 1; rx_pos = 0;
    } else {
        if (rx_pos < 18) rx_buffer[rx_pos++] = data;
    }
}

// --- УПРАВЛЕНИЕ МОТОРОМ ---

void set_motor(uint8_t state) {
    fan_on = state;
    eeprom_update_byte(&ee_fan_on, state);
    if (state) {
        MOTOR_PORT |= (1 << IN1_PIN);
        MOTOR_PORT &= ~(1 << IN2_PIN);
        OCR0 = fan_speed;
    } else {
        MOTOR_PORT &= ~((1 << IN1_PIN) | (1 << IN2_PIN));
        OCR0 = 0;
    }
    update_lcd_needed = 1;
}

// --- ОБРАБОТЧИК ПРЕРЫВАНИЯ КНОПОК INT0 ---

void init_int0(void) {
    // Настраиваем PD2 (INT0) на вход с подтяжкой
    DDRD &= ~(1 << INT0_PIN);
    PORTD |= (1 << INT0_PIN);

    // Настройка прерывания: Falling Edge (спадающий фронт -
    нажатие кнопки)
    // ISC01 = 1, ISC00 = 0 в регистре MCUCR
    MCUCR |= (1 << ISC01);
    MCUCR &= ~(1 << ISC00);

    // Разрешаем внешнее прерывание INT0
    GICR |= (1 << INT0);
}
```

Продолжение листинга А.1

```
ISR(INT0_vect) {
    // Простой программный антидребезг прямо в прерывании
    _delay_ms(30);

    // Проверяем, какая кнопка нажата на PORTA
    // (Логика нажатия: 0 на пине)

    // Кнопка AUTO
    if (!(BTN_PIN & (1 << BTN_AUTO))) {
        mode_auto = 1;
        eeprom_update_byte(&ee_mode_auto, 1);
        uart_send_str_P(PSTR("INT: AUTO Mode\r\n"));
        update_lcd_needed = 1;
    }
    // Кнопка TOGGLE (Manual ON/OFF)
    else if (!(BTN_PIN & (1 << BTN_TOGGLE))) {
        mode_auto = 0;
        eeprom_update_byte(&ee_mode_auto, 0);
        set_motor(!fan_on); // Переключаем состояние
        uart_send_str_P(PSTR("INT: Toggle Man\r\n"));
        update_lcd_needed = 1;
    }
    // Кнопка UP
    else if (!(BTN_PIN & (1 << BTN_UP))) {
        int new_speed = fan_speed + 25;
        if (new_speed >= 255) new_speed = 255;
        fan_speed = (uint8_t)new_speed;
        eeprom_update_byte(&ee_fan_speed, fan_speed);
        if (fan_on) OCR0 = fan_speed;

        uart_send_str_P(PSTR("INT: Speed UP: "));
        uart_send_num(fan_speed); uart_send_str_P(PSTR("\r\n"));
        update_lcd_needed = 1;
    }
    // Кнопка DOWN
    else if (!(BTN_PIN & (1 << BTN_DOWN))) {
        int new_speed = fan_speed - 25;
        if (new_speed <= 0) new_speed = 0;
        fan_speed = (uint8_t)new_speed;
        eeprom_update_byte(&ee_fan_speed, fan_speed);
        if (fan_on) OCR0 = fan_speed;

        uart_send_str_P(PSTR("INT: Speed DOWN: "));
        uart_send_num(fan_speed); uart_send_str_P(PSTR("\r\n"));
        update_lcd_needed = 1;
    }
}
```

Продолжение листинга А.1

```
// Ждем, пока кнопка будет отпущена, чтобы не вызывать
прерывание повторно слишком часто
// (опционально, но полезно для предотвращения дребезга
при отпускании)
// while(!(PIND & (1 << INT0_PIN)));

// Очистка флага INTF0 происходит автоматически при выходе
из ISR
}

// --- DHT ДАТЧИК ---

uint8_t dht_read(void) {
    uint8_t bits[5]; uint8_t cnt = 7; uint8_t idx = 0;
    for(int k=0; k<5; k++) bits[k] = 0;

    DHT_DDR |= (1 << DHT_PIN); DHT_PORT &= ~(1 << DHT_PIN);
    _delay_ms(20); DHT_PORT |= (1 << DHT_PIN); DHT_DDR &= ~(1
    << DHT_PIN);

    _delay_us(40); if ((DHT_PIN_IN & (1 << DHT_PIN))) return
1;
    _delay_us(80); if (!(DHT_PIN_IN & (1 << DHT_PIN))) return
2;
    _delay_us(80);

    for (int i = 0; i < 40; i++) {
        uint8_t timeout = 0;
        while(!(DHT_PIN_IN & (1 << DHT_PIN))) { _delay_us(1);
if (++timeout > 100) return 3; }
        _delay_us(30);
        if (DHT_PIN_IN & (1 << DHT_PIN)) {
            if (idx < 5) bits[idx] |= (1 << cnt);
            timeout = 0;
            while(DHT_PIN_IN & (1 << DHT_PIN)) { _delay_us(1);
if (++timeout > 100) return 4; }
        }
        if (cnt == 0) { cnt = 7; idx++; } else { cnt--; }
    }
    if ((uint8_t)(bits[0] + bits[1] + bits[2] + bits[3]) ==
bits[4]) {
        humidity = bits[0]; temp_c = bits[2]; return 0;
    }
    return 5;
}
```

Продолжение листинга А.1

```
void update_lcd_info(void) {
    lcd_gotoxy(0, 0);
    lcd_str_P(PSTR("T:")); lcd_num(temp_c); lcd_char(0xDF);
    lcd_str_P(PSTR("C H:")); lcd_num(humidity);
    lcd_str_P(PSTR("% ")); lcd_char(mode_auto ? 'A' : 'M');

    lcd_gotoxy(0, 1);
    if (fan_on) {
        lcd_str_P(PSTR("ON Spd:")); lcd_num(fan_speed);
        lcd_str_P(PSTR(" "));
    } else {
        lcd_str_P(PSTR("OFF H:")); lcd_num(temp_high);
        lcd_str_P(PSTR(" L:")); lcd_num(temp_low);
    }
}

// --- MAIN ---

int main(void) {
    MOTOR_DDR |= (1 << IN1_PIN) | (1 << IN2_PIN);
    PWM_DDR |= (1 << PWM_PIN);

    // Fast PWM
    TCCR0 = (1 << WGM00) | (1 << WGM01) | (1 << COM01) | (1 <<
CS01);
    OCR0 = 0;

    // Настройка кнопок на вход с подтяжкой (PORTA)
    BTN_DDR &= ~(1 << BTN_AUTO) | (1 << BTN_TOGGLE) | (1 <<
BTN_UP) | (1 << BTN_DOWN));
    BTN_PORT |= (1 << BTN_AUTO) | (1 << BTN_TOGGLE) | (1 <<
BTN_UP) | (1 << BTN_DOWN);

    uart_init_fixed();
    load_config();
    lcd_init();

    set_motor(fan_on);

    // ИНИЦИАЛИЗАЦИЯ ПРЕРЫВАНИЯ КНОПОК
    init_int0();

    sei(); // Разрешить глобальные прерывания

    uart_send_str_P(PSTR("System Ready (INT0 Mode)\r\n"));
}
```

Продолжение листинга A.1

```
    lcd_cmd(0x01); lcd_gotoxy(0, 0); lcd_str_P(PSTR("System
Ready"));
    _delay_ms(1000); lcd_cmd(0x01);

    uint8_t timer_ticks = 0;

    // Главный цикл стал чище, так как кнопки обрабатываются в
    ISR
    while (1) {
        // Обработка команд UART
        if (cmd_ready) {
            _delay_ms(5);
            char *cmd = (char*)rx_buffer;

            if (cmd[0] == '1') {
                mode_auto = 0;
                eeprom_update_byte(&ee_mode_auto, 0);
                set_motor(1);
                uart_send_str_P(PSTR("Manual ON\r\n"));
            }
            else if (cmd[0] == '0') {
                mode_auto = 0;
                eeprom_update_byte(&ee_mode_auto, 0);
                set_motor(0);
                uart_send_str_P(PSTR("Manual OFF\r\n"));
            }
            else if (cmd[0] == 'A' || cmd[0] == 'a') {
                mode_auto = 1;
                eeprom_update_byte(&ee_mode_auto, 1);
                uart_send_str_P(PSTR("Mode: AUTO\r\n"));
                update_lcd_needed = 1;
            }
            else if (cmd[0] == 'M' || cmd[0] == 'm') {
                mode_auto = 0;
                eeprom_update_byte(&ee_mode_auto, 0);
                uart_send_str_P(PSTR("Mode: MANUAL\r\n"));
                update_lcd_needed = 1;
            }
            else if (cmd[0] == 'V' || cmd[0] == 'v') {
                int val = atoi(&cmd[1]);
                if (val >= 0 && val <= 255) {
                    fan_speed = val;
                    eeprom_update_byte(&ee_fan_speed, val);
                    if(fan_on) OCR0 = fan_speed;
                    update_lcd_needed = 1;
                }
            }
        }
    }
```

Продолжение листинга А.1

```
    }
    else if (cmd[0] == 'H' || cmd[0] == 'h') {
        int val = atoi(&cmd[1]);
        if (val > temp_low && val < 99) {
            temp_high = val;
eeprom_update_byte(&ee_temp_high, val);
            update_lcd_needed = 1;
        }
    }
    else if (cmd[0] == 'L' || cmd[0] == 'l') {
        int val = atoi(&cmd[1]);
        if (val > 0 && val < temp_high) {
            temp_low = val;
eeprom_update_byte(&ee_temp_low, val);
            update_lcd_needed = 1;
        }
    }
    cmd_ready = 0;
}

// Таймер для опроса датчика и обновления экрана
_delay_ms(100);
timer_ticks++;
if (timer_ticks >= 20) { // ~2 секунды
    timer_ticks = 0;
    if (dht_read() == 0) {
        update_lcd_needed = 1;
        if (mode_auto) {
            if (temp_c >= temp_high && !fan_on)
set_motor(1);
            else if (temp_c <= temp_low && fan_on)
set_motor(0);
        }
    }
}

if (update_lcd_needed) {
    update_lcd_info();
    update_lcd_needed = 0;
}
}
```